CONFIDENTIAL · INTERNAL USE ONLY

Arthaleads CRM
Deep Audit Report

A comprehensive security, bug, and architectural review of the Arthaleads CRM platform — covering authentication vulnerabilities, tenant isolation gaps, 500 error locations, missing features, and the login session-expired race condition.

DATE	4 June 2026
PROJECT	prophuntllp-bit / arthaleads
STACK	Node.js · Express · MongoDB · React · Vite · Railway
SCOPE	Authentication · Multi-tenancy · Webhooks · API Security · XSS · 500 Errors · Race Conditions · Missing Features
FINDINGS	2× P0 Critical · 4× P1 High · 5× P2 Medium · 4× P3 Low

CONTENTS

What's Inside

PART 1

The Login → "Session Expired" Bug

⚠️ **SYMPTOM**

User logs in successfully, the dashboard renders, then within 1–3 seconds the user is kicked out with "Your session has expired. Please log in again." Happens on some devices (iOS Safari, mobile Chrome in Private Mode) reliably; intermittent on desktop.

How the Auth Flow Works

On mount, `AuthContext.jsx` fires `GET /auth/me` with an `AbortController` attached. When the user submits the login form, `login()` calls `authMeControllerRef.current?.abort()` to cancel the pending `/auth/me`, then calls `POST /auth/login`. If the abort fires before Axios processes the response, the cancel is silent. If not, the race condition fires.

Scenario A — Railway Cold-Start Race

Railway containers sleep after inactivity. A cold boot takes 15–25 s. The `/auth/me` request is sent on mount. The user types credentials and logs in. `login()` fires `abort()`. But if Railway hasn't responded to `/auth/me` yet, the network response can arrive after the AbortController fires but before Axios processes the cancel signal. Axios then emits the 401 error response, the interceptor in `api.js` dispatches `auth:expired`, localStorage is cleared, and the user is kicked out within seconds of logging in.

Scenario B — iOS Safari / Private Mode (highest real-world impact)

The backend is on `api.arthaleads.com`, the frontend on `www.arthaleads.com`. The cookie uses `SameSite=None; Secure; Domain=.arthaleads.com`. iOS Safari 16+ and all Private Browsing tabs treat this as a third-party cookie and **reject it silently**.

The Bearer token fallback (`_at` in `localStorage`) should cover this. But in Private Mode, `localStorage.setItem()` either throws `QuotaExceededError` silently or appears to succeed but discards the value on next read. So: login succeeds, token is not stored, page reloads or `/auth/me` fires, returns 401, session-expired toast fires.

🔴 **THIS IS THE MOST COMMON REAL-WORLD CAUSE ON MOBILE DEVICES**

iOS Safari in standard mode also applies ITP (Intelligent Tracking Prevention) which can evict localStorage data after 7 days of no interaction with the site. A user who hasn't opened the CRM in a week on iPhone will always see session expired.

Scenario C — Domain / URL Mismatch

If the Railway-generated URL (`something.up.railway.app`) is used as the API base URL instead of `api.arthaleads.com`, the `Domain=.arthaleads.com` cookie attribute is silently ignored by the browser (wrong eTLD+1). The cookie lands only on the Railway domain. The frontend on `arthaleads.com` never sends it. The `_at` localStorage fallback handles this — but only if `VITE_API_URL` is set to the same Railway URL consistently in production.

Scenario D — Token Not Persisted Before Parallel Request

`persist()` in `AuthContext.jsx` calls `localStorage.setItem("_at", token)`. The `api.js` request interceptor reads `localStorage.getItem("_at")` synchronously on every request. If any component fires a parallel API call in the same tick that `login()` resolves (e.g., a `useEffect` triggered by the user state update before `persist()` completes its synchronous writes), that call goes out without the Bearer token, may return 401, and triggers the interceptor.

The Definitive Fix (3 independent patches)

Fix 1 — iOS Safari / Private Mode (highest impact): Wrap `localStorage.setItem` in a try/catch in `persist()`. If localStorage throws or is unavailable, store the token in `sessionStorage` instead. Update the `api.js` request interceptor to check both stores.

```
// AuthContext.jsx — persist()
const saveToken = (token) => {
  try {
    localStorage.setItem("_at", token);
  } catch {
    try { sessionStorage.setItem("_at", token); } catch {}
  }
};

// api.js — request interceptor
const t = localStorage.getItem("_at") || sessionStorage.getItem("_at");
if (t) config.headers.Authorization = `Bearer ${t}`;
```

Fix 2 — AbortController race (cold-start): Set a boolean flag when login is in flight so the `auth:expired` event listener can ignore 401s that arrive during login.

```
// AuthContext.jsx
const loggingInRef = useRef(false);

const login = useCallback(async (email, password) => {
  loggingInRef.current = true;
  authMeControllerRef.current?.abort();
  try {
    const { data } = await api.post("/auth/login", { email, password });
    persist(data.user, data.org, data.token);
    return data;
  } finally {
    loggingInRef.current = false;
  }
}, [persist]);

// In the auth:expired handler:
const onExpired = () => {
  if (loggingInRef.current) return; // ignore 401 during login
  clearSession(); setLoading(false);
};
```

Fix 3 — Domain / URL consistency: Confirm that `VITE_API_URL` in the production Railway environment points to `https://api.arthaleads.com` and that the backend cookie sets `domain: ".arthaleads.com"`. These must match. Document this pairing in `.env.example`.

PART 2 — SECURITY

Critical Findings (P0)

🔴 **P0 — FIX BEFORE NEXT PRODUCTION DEPLOY**

Both findings below can be fixed in under 30 minutes total. P0-1 is a guaranteed 500 crash on a code path. P0-2 allows arbitrary fake leads to be injected into any tenant's pipeline without authentication.

P0-1 CRITICAL verifySystemToken crashes with ReferenceError → 500

`backend/controllers/automationController.js · ~line 100 (verifySystemToken function)`

`AppError` is used inside `verifySystemToken()` to throw an authentication error, but `AppError` is **never imported** at the top of the file. When any request reaches this code path with an empty or missing system token, Node.js throws `ReferenceError: AppError is not defined`. This is an unhandled synchronous exception inside an async function. Express 4 does not auto-catch synchronous throws in async route handlers — it reaches the global uncaught exception handler and calls `process.exit(1)`, taking the entire server down.

Any admin who triggers an automation action requiring system token verification will crash the production server.

FIX — 5 MINUTES

Add to the top of `automationController.js`:

```
const { AppError } = require("../middlewares/errorHandler");
```

P0-2 CRITICAL Missing FB_APP_SECRET silently disables webhook signature verification

`backend/routes/webhookRoutes.js · lines 20–26 (verifyFbSignature)`

The `verifyFbSignature` function checks whether `process.env.FB_APP_SECRET` is set. If it is absent, the function logs a warning and **returns without throwing**. This means every incoming `POST /webhook` request is accepted as valid regardless of its content or origin.

If `FB_APP_SECRET` is accidentally missing from the Railway environment (e.g. after a re-deploy, variable reset, or environment clone), any HTTP client on the internet can POST arbitrary payloads to `/webhook` and create fake leads in *any tenant's* CRM pipeline — with any name, phone number, or content — with zero authentication.

FIX — 20 MINUTES

In `verifyFbSignature`, change the missing-secret branch to throw in production: `if (!process.env.FB_APP_SECRET && process.env.NODE_ENV === "production") { const err = new Error("FB_APP_SECRET not configured"); err.status = 503; throw err; }`

Also add a startup assertion in `server.js`: if NODE_ENV is production and FB_APP_SECRET is absent, log a fatal error at boot.

High Severity Findings (P1)

Fix within the next sprint. Each finding has a clear exploit path and a one-line or short-function fix.

P1-1 HIGH Google auth does not verify email_verified flag

`backend/services/authService.js · lines 177–237 (googleAuth)`

The `/oauth2/v3/userinfo` endpoint returns an `email_verified` boolean alongside the user's profile. The current code destructures `{ sub, email, name, picture }` and ignores this field entirely.

Google issues OAuth tokens for unverified email addresses in some flows (e.g. Google Workspace accounts where the domain administrator has disabled email verification). An attacker who controls an unverified Google account with the email `victim@example.com` can use the Google sign-in flow to link that Google identity to an existing CRM user's account (the code path at line 198: "If found by email but no googleId yet, link the Google account"). This gives them full access to the victim's CRM account, all their org's leads, settings, and data.

FIX — 10 MINUTES

After destructuring the `payload`, add: `if (!payload.email_verified) throw new AppError("Google account email is not verified. Please verify your Google account first.", 400);`

P1-2 HIGH Facebook token refresh endpoint has no org boundary check

`backend/routes/automationRoutes.js · token-refresh route`

The token refresh endpoint queries the automation by `{ _id: automationId, platform: "Facebook", isActive: true }` — there is no `orgId: req.user.orgId` filter. Any authenticated admin or manager from *any* organisation can supply a foreign automation's MongoDB ObjectId and trigger a Facebook page token refresh for another tenant's connected Facebook page.

While this doesn't directly expose data, it can silently overwrite another tenant's stored access token with a refreshed version, potentially disrupting their Facebook lead pipeline. It also means tenant A can probe whether tenant B has a Facebook automation connected by iterating ObjectIds.

FIX — 10 MINUTES

Add `orgId: req.user.orgId` to the automation lookup query. This ensures only automations belonging to the authenticated user's org can be refreshed.

P1-3 HIGH Stored XSS via document.write() in Performance PDF export

`frontend/src/pages/Performance.jsx · exportPDF() function`

The `exportPDF()` function builds a raw HTML string by directly interpolating user-controlled values — `orgName`, `m.name`, `m.email` — without any escaping, then passes the string to `win.document.write(html)` in a popup window at the same origin.

Any user with permission to edit their display name or any admin who can set the org name via Settings can inject arbitrary HTML and JavaScript. Because the popup is opened at the same origin as the CRM app, injected scripts have full access to the DOM, cookies (non-httpOnly), and can perform any action the logged-in user can perform — including reading all leads, exporting data, or creating users. The payload persists in the database and fires for every admin who exports the performance report.

FIX — 1 HOUR

Replace string interpolation with DOM construction: use `document.createElement()` and `element.textContent = value` instead of injecting into an HTML string. Alternatively, run the HTML string through `DOMPurify.sanitize(html)` before writing it.

P1-4 HIGH **Attendance adminEntry: userId not validated against org**

`backend/controllers/attendanceController.js · lines 241–285 (adminEntry)`

The upsert filter includes `orgId: req.user.orgId` which correctly prevents modifying an *existing* attendance record for another org. However, on an upsert where no record exists, MongoDB creates a new document with the attacker-supplied `userId` and the authenticated user's `orgId`. There is no check that the `userId` in the request body actually belongs to `req.user.orgId`.

A malicious admin can supply any valid MongoDB ObjectId as `userId` — including a super_admin user ID or a user from another organisation — and fabricate attendance records for that user, scoped to their own org. The subsequent `.populate("userId", "name email role")` will resolve the foreign user's real details into the response.

FIX — 20 MINUTES

Before the `findOneAndUpdate`, add a membership check:

```
const targetUser = await User.findOne({ _id: userId, orgId: req.user.orgId });
if (!targetUser) return next(new AppError("User not found in your organisation", 404));
```

Medium Severity Findings (P2)

P2-1 MEDIUM Facebook access tokens stored as plaintext in MongoDB

`backend/models/Automation.js · accessToken, userToken fields`

Both `accessToken` (Facebook page token) and `userToken` (long-lived user token) are stored as plain `String` fields in MongoDB. Facebook long-lived user tokens have a 60-day lifetime and carry permissions to read lead data from any page the user manages. If the MongoDB Atlas cluster is ever compromised — via leaked connection string, misconfigured Atlas network access, or a database dump — all tokens are exposed in cleartext.

FIX — 4 HOURS

Encrypt at rest using AES-256-GCM with a `CRYPTO_KEY` env var. Add Mongoose virtual getters/setters that transparently encrypt on write and decrypt on read. At minimum: restrict Atlas IP allowlist to Railway's static IPs and enable Atlas Encryption at Rest.

P2-2 MEDIUM OTP verification has no attempt counter — brute-forceable with rotating IPs

`backend/controllers/authController.js · signupVerifyOtp (lines 263–292)`

A 6-digit OTP has 1,000,000 combinations. The only protection is the IP-based rate limiter (50 req / 15 min / IP). An attacker with a pool of rotating proxy IPs (cheap and widely available) can try thousands of combinations per minute across IPs and brute-force a target's OTP before its 5-minute expiry, especially for a known phone number.

FIX — 1 HOUR

Add an `attempts` counter field to `SignupOtp`. Increment on each failed check. After 5 failed attempts, delete the record and return a 429 forcing the user to request a new OTP. This limits brute force to 5 guesses per OTP regardless of IP rotation.

P2-3 MEDIUM No CSRF protection on cookie-based auth endpoints

`backend/server.js · CORS + cookie configuration`

The app uses `SameSite=None` cookies (required for cross-subdomain delivery). `SameSite=None` does not provide any CSRF protection — it explicitly allows cross-site requests to carry the cookie. The CORS `allowedOrigins` list prevents cross-origin *reads* of responses, but CORS preflight does not block state-mutating POST/PATCH requests from being sent and processed on the server. A CSRF attack does not need to read the response — it just needs the server to process the request.

FIX — 2 HOURS

Implement Double Submit Cookie CSRF protection: generate a random token, store it in a non-httpOnly cookie (`crm_csrf`), and require it as a header (`X-CSRF-Token`) on all state-mutating requests. The server compares header vs. cookie value. Alternatively, verify `req.headers.origin` against the allowlist on all POST/PATCH/DELETE routes.

P2-4 MEDIUM Ticket attachment size limit defined but not enforced

`backend/controllers/ticketController.js · sanitiseAttachments, MAX_ATTACH_BYTES`

`MAX_ATTACH_BYTES = 600 * 1024` (600 KB) is defined but `sanitiseAttachments` only truncates the URL string at 2,000,000 characters. It never checks `a.size` against `MAX_ATTACH_BYTES`. A client can submit ticket attachments with any reported size and the server stores them without validation.

FIX — 15 MINUTES

In `sanitiseAttachments`, add: `if (a.size && a.size > MAX_ATTACH_BYTES) throw new AppError(`Attachment "${a.name}" exceeds the 600 KB limit`, 400);`

P2-5 MEDIUM **Single global Voice API key — leaked key exposes all tenant data**

`backend/middlewares/apiKey.js · backend/routes/voiceRoutes.js`

One `VOICE_API_KEY` env var authenticates all voice API requests. The org scope is determined by the caller-supplied `X-Org-Id` header (or `?org_id` query param or `VOICE_ORG_ID` env). The voice routes correctly scope reads/writes to `req.voiceOrgId`. However, if the single API key is ever leaked, a holder of that key can access any tenant's leads by passing different `X-Org-Id` values — no additional authentication is required.

FIX — 3 HOURS

Issue per-org API keys stored in the `Organization` document. The middleware looks up the key in the DB, finds the org, and sets `req.voiceOrgId` from the database record rather than from the caller-supplied header. This eliminates both the single point of failure and the caller-controlled org spoofing.

Low Severity / Informational (P3)

P3-1 LOW Facebook webhook scans all orgs — performance + tenant isolation risk

`backend/routes/webhookRoutes.js · findFacebookAutomationByPayload (lines 74–100)`

The initial `Automation.find()` has no `orgId` filter. Every incoming Facebook webhook POST scans *all* active Facebook automations across all tenants. As tenant count grows this becomes a performance hotspot. Additionally, if two tenants have automations for the same Facebook page, the first automation sorted by `updatedAt desc` wins — potentially routing leads to the wrong org.

RECOMMENDATION

Add a MongoDB index on `{ platform: 1, isActive: 1, pageId: 1 }`. Restructure the query to filter by `pageId` directly in the initial `find()` since `leadData.page_id` is available at that point.

P3-2 LOW No request correlation ID or persistent admin audit log

Failed sensitive operations (failed login, password reset, token refresh) are logged to Railway but have no per-request correlation ID. Tracing a specific user's failed login across 50,000 log lines requires timestamp-based grep. Railway log retention is limited. Admin actions (user creation/deactivation, org plan changes, data deletion) leave no persistent database-level audit trail.

RECOMMENDATION

Add a `x-request-id` middleware (one UUID per request). Log and return in response headers. Add an `AuditLog` Mongoose model for sensitive admin actions (who did what to whom, when, from which IP).

P3-3 LOW Password minimum is 6 characters with no complexity requirement

`backend/controllers/authController.js · line 333`

The only password rule is `password.length < 6`. NIST SP 800-63B recommends a minimum of 8 characters. There is no complexity requirement, no common-password check, and no breach-list check. A user can set the password `123456` or `aaaaaa`.

RECOMMENDATION

Increase minimum to 8 characters. Add frontend password-strength scoring via `zxcvbn` (shows strength meter, blocks obviously weak passwords client-side). No backend complexity rules needed beyond the length increase per NIST guidelines.

P3-4 LOW Public /api/error-report endpoint covered only by the general rate limiter

`backend/server.js · lines 205–216`

`POST /api/error-report` is unauthenticated and only protected by the general 200 req/15 min rate limiter. It forwards content directly to Sentry. An attacker can flood this endpoint to exhaust Sentry's monthly event quota, effectively blinding the team to real errors in production.

RECOMMENDATION

Apply `contactLimiter` (10 req/15 min/IP) to this endpoint, or require a static `X-Error-Token` header set at Vite build time that the server validates.

PART 3

Known 500 Error Locations

Location	Trigger	Type	Impact
`automationController.js` `verifySystemToken`	Empty / missing system token in request	HARD CRASH	ReferenceError → `process.exit(1)` — server restarts
`webhookRoutes.js` POST handler	Unexpected non-auth error during lead fetch	Graceful 500	Returns `{ success:false, message: "Webhook processing failed" }` — no crash
`authController.js` `signupSendOtp`	Resend API down or `RESEND_API_KEY` missing	Graceful 500	Caught, wrapped in AppError — returns 500 to client
`server.js` `connectDB()`	MongoDB Atlas unreachable at boot	Intentional exit	`process.exit(1)` — correct, Railway will restart
Any async route without `try/catch`	Unhandled Promise rejection	POTENTIAL CRASH	Express 4 does not auto-catch async throws — reaches `uncaughtException` handler → exit(1)
Mongoose validation errors	Invalid data submitted to any create/update route	Unformatted 500	Global error handler does not specifically format `ValidationError` — falls through as generic 500 instead of 400 with field messages

💡 **QUICK WIN — MONGOOSE VALIDATIONERROR FORMATTING**

In `middlewares/errorHandler.js`, add a case for `err.name === "ValidationError"`: extract field-level messages from `err.errors`, return a 400 with a `fields` object. This turns cryptic 500s on form submissions into actionable 400 validation errors.

PART 4

Missing Features & Architectural Gaps

Authentication & Account Security

- **No email verification on signup.** Users can register with any email address. Typos result in users who can never receive password-reset or notification emails. An attacker can register with `victim@company.com` and create an org in that name.
- **No session revocation / "sign out all devices" feature.** The JWT has a 30-day lifetime. There is no token blacklist. A stolen JWT cannot be invalidated short of changing `JWT_SECRET`, which logs out every user on the platform.
- **No account lockout after repeated failed password attempts.** The IP rate limiter (50 req / 15 min) is the only protection. Distributed password-spray attacks are not mitigated.
- **No 2FA / TOTP option.** High-privilege admin accounts have no second factor.

Multi-tenancy & Data Isolation

- **No per-tenant data caps.** A single tenant can grow leads, attachments, or AI usage without limit, affecting query performance for all other tenants on the shared MongoDB cluster.
- **No database migration tracking.** Migrations run inline in `server.js` `connectDB().then()` with no record of completion. A failed partial migration runs again on the next boot.

Integrations

- **No 99acres / Housing.com / MagicBricks integration.** All three portals send leads via email (configurable forwarding address). The recommended medium-term path is an email-parse webhook: configure a dedicated inbound email address (via Resend Inbound or Mailgun Routes), parse the standardised portal email format server-side, and create Lead documents automatically.

Operational

- **Health check does not verify DB connectivity.** `GET /health` returns `{ status: "ok" }` even if MongoDB is disconnected. Load balancers and uptime monitors that rely on this endpoint will not detect a DB outage.
- **No persistent admin audit trail.** There is no database record of who created/deactivated users, changed org plans, or deleted data — only Railway logs with limited retention.

SUMMARY

Priority Table — Complete Findings

ID	Severity	Finding	File	Effort
Login	P0 BUG	iOS Safari / Private Mode drops `_at` token → instant session expired	`AuthContext.jsx`, `api.js`	2 hrs
P0-1	CRITICAL	`AppError` not imported → ReferenceError → server crash	`automationController.js`	5 min
P0-2	CRITICAL	Missing `FB_APP_SECRET` silently disables webhook auth	`webhookRoutes.js`	30 min
P1-1	HIGH	Google auth skips `email_verified` — account takeover risk	`authService.js`	10 min
P1-2	HIGH	Facebook token refresh has no org boundary — cross-tenant access	`automationRoutes.js`	10 min
P1-3	HIGH	XSS via `document.write()` in Performance PDF export	`Performance.jsx`	1 hr
P1-4	HIGH	Attendance `adminEntry`: `userId` not validated against org	`attendanceController.js`	20 min
P2-1	MEDIUM	Facebook tokens stored as plaintext in MongoDB	`Automation.js`	4 hrs
P2-2	MEDIUM	OTP brute-forceable — no attempt counter	`authController.js`	1 hr
P2-3	MEDIUM	No CSRF protection on `SameSite=None` cookie endpoints	`server.js`	2 hrs
P2-4	MEDIUM	Ticket attachment size limit defined but not enforced	`ticketController.js`	15 min
P2-5	MEDIUM	Single global Voice API key — leaked key exposes all tenants	`apiKey.js`, `voiceRoutes.js`	3 hrs
P3-1	LOW	FB webhook scans all orgs — performance + tenant isolation risk	`webhookRoutes.js`	1 hr
P3-2	LOW	No request correlation ID or persistent admin audit log	Multiple files	4 hrs
P3-3	LOW	Password minimum 6 chars, no complexity requirement	`authController.js`	15 min
P3-4	LOW	Public `/api/error-report` can flood Sentry quota	`server.js`	10 min

Recommended Immediate Action Plan (Sprint 1)

The following 6 items can be completed in under 3 hours total and close the highest-risk attack surfaces:

1. **P0-1** — Add missing `AppError` import to `automationController.js` (5 min)
2. **P1-1** — Add `email_verified` check to Google auth (10 min)
3. **P1-2** — Add `orgId` filter to Facebook token refresh route (10 min)
4. **P1-4** — Validate `userId` belongs to org in `adminEntry` (20 min)
5. **Login fix** — Wrap `localStorage.setItem` in try/catch with `sessionStorage` fallback (1 hr)
6. **P0-2** — Add production guard when `FB_APP_SECRET` is absent (30 min)

✅ **WHAT'S ALREADY DONE WELL**

httpOnly cookie auth + Bearer token fallback architecture is solid. AbortController race condition fix is correctly implemented. Rate limiters are well-configured with separate limits for auth, contact, blog, and general endpoints. CORS allowlist is env-driven and correct. Org-level access guard with in-memory cache (60s TTL) is efficient. Graceful shutdown handles SIGTERM correctly for Railway deploys.